



100 Python Programs Compulsory Before You Start Data Science

Every program written out in full, verified working, with a one-line description of what it solves. Read the description, try writing it yourself, then compare.

10 Basics (1–20)

01 — Hello World. The first program everyone writes.

```
print("Hello, World!")
```

02 — Swap two variables without a temporary variable.

```
a, b = 5, 10
a, b = b, a
print(a, b) # 10 5
```

03 — Check even or odd.

```
n = 7
result = "Even" if n % 2 == 0 else "Odd"
print(result)
```

04 — Find the largest of three numbers.

```
x, y, z = 3, 9, 6
print(max(x, y, z))
```

05 — Check if a number is prime.

```
def is_prime(n):
    if n < 2:
        return False
    for i in range(2, int(n**0.5) + 1):
        if n % i == 0:
            return False
    return True

print(is_prime(17))
```

06 — Print the Fibonacci series.

```
def fibonacci(n):
    a, b = 0, 1
    seq = []
    for _ in range(n):
        seq.append(a)
        a, b = b, a + b
    return seq

print(fibonacci(10))
```

07 — Calculate a factorial.

```
def factorial(n):
    return 1 if n == 0 else n * factorial(n - 1)

print(factorial(5))
```

08 — Check if a number is a palindrome.

```
def is_palindrome_num(n):
    return str(n) == str(n)[::-1]

print(is_palindrome_num(121))
```

09 — Reverse a number.

```
def reverse_number(n):
    return int(str(n)[::-1])

print(reverse_number(1234))
```

10 — Sum the digits of a number.

```
def sum_of_digits(n):
    return sum(int(d) for d in str(n))

print(sum_of_digits(12345))
```

11 — Check for an Armstrong number.

```
def is_armstrong(n):
    digits = str(n)
    power = len(digits)
    return n == sum(int(d) ** power for d in digits)

print(is_armstrong(153))
```

12 — Convert Celsius to Fahrenheit.

```
def celsius_to_fahrenheit(c):
    return (c * 9 / 5) + 32

print(celsius_to_fahrenheit(100))
```

13 — Build a simple calculator.

```
def calculator(a, b, op):
    if op == '+': return a + b
    elif op == '-': return a - b
    elif op == '*': return a * b
    elif op == '/': return a / b

print(calculator(6, 3, '+'))
```

14 — Check if a year is a leap year.

```
def is_leap_year(y):  
    return y % 4 == 0 and (y % 100 != 0 or y % 400 == 0)  
  
print(is_leap_year(2024))
```

15 — Find the GCD of two numbers.

```
def gcd(a, b):  
    while b:  
        a, b = b, a % b  
    return a  
  
print(gcd(48, 18))
```

16 — Find the LCM of two numbers.

```
def lcm(a, b):  
    return a * b // gcd(a, b)  
  
print(lcm(4, 6))
```

17 — Print a multiplication table.

```
def multiplication_table(n):  
    return [n * i for i in range(1, 11)]  
  
print(multiplication_table(5))
```

18 — Sum the first N natural numbers.

```
def sum_natural(n):  
    return n * (n + 1) // 2  
  
print(sum_natural(10))
```

19 — Count the digits in a number.

```
def count_digits(n):
    return len(str(abs(n)))

print(count_digits(12345))
```

20 — Check if a number is a perfect number.

```
def is_perfect(n):
    return n == sum(i for i in range(1, n) if n % i == 0)

print(is_perfect(28))
```

Strings (21–35)

21 — Reverse a string.

```
def reverse_string(s):
    return s[::-1]

print(reverse_string("hello"))
```

22 — Check if a string is a palindrome.

```
def is_palindrome_str(s):
    return s == s[::-1]

print(is_palindrome_str("madam"))
```

23 — Count the vowels in a string.

```
def count_vowels(s):
    return sum(1 for c in s.lower() if c in 'aeiou')

print(count_vowels("Hello World"))
```

24 — Count the words in a sentence.

```
def count_words(s):
    return len(s.split())

print(count_words("the quick brown fox"))
```

25 — Check if two strings are anagrams.

```
def is_anagram(s1, s2):
    return sorted(s1.lower()) == sorted(s2.lower())

print(is_anagram("listen", "silent"))
```

26 — Remove duplicate characters from a string.

```
def remove_duplicates_str(s):
    seen = set()
    result = []
    for c in s:
        if c not in seen:
            seen.add(c)
            result.append(c)
    return ''.join(result)

print(remove_duplicates_str("programming"))
```

27 — Find the most frequent character in a string.

```
def most_frequent_char(s):
    return max(set(s), key=s.count)

print(most_frequent_char("aabbcc"))
```

28 — Convert a string to uppercase and lowercase.

```
def case_convert(s):
    return s.upper(), s.lower()
```

```
print(case_convert("Hello"))
```

29 — Count occurrences of a substring.

```
def count_substring(s, sub):  
    return s.count(sub)  
  
print(count_substring("abcabcabc", "abc"))
```

30 — Check if a string is a pangram.

```
def is_pangram(s):  
    return set('abcdefghijklmnopqrstuvwxyz').issubset(set(s.lower()))  
  
print(is_pangram("The quick brown fox jumps over the lazy dog"))
```

31 — Title-case every word in a sentence.

```
def title_case(s):  
    return s.title()  
  
print(title_case("the great gatsby"))
```

32 — Replace a character throughout a string.

```
def replace_char(s, old, new):  
    return s.replace(old, new)  
  
print(replace_char("banana", "a", "o"))
```

33 — Split a string into a list of words.

```
def split_words(s):  
    return s.split()
```

```
print(split_words("one two three"))
```

34 — Join a list of strings into one.

```
def join_words(lst):  
    return ' '.join(lst)  
  
print(join_words(["one", "two", "three"]))
```

35 — Check if a string contains only digits.

```
def only_digits(s):  
    return s.isdigit()  
  
print(only_digits("12345"))
```

Lists (36–50)

36 — Find the largest element in a list.

```
def largest(lst):  
    return max(lst)  
  
print(largest([3, 7, 2, 9, 4]))
```

37 — Find the second largest element in a list.

```
def second_largest(lst):  
    return sorted(set(lst))[-2]  
  
print(second_largest([3, 7, 2, 9, 4]))
```

38 — Reverse a list without using built-ins.

```
def reverse_list(lst):  
    return lst[::-1]
```



```
print(reverse_list([1, 2, 3]))
```

39 — Remove duplicates from a list.

```
def remove_dup_list(lst):  
    return list(dict.fromkeys(lst))  
  
print(remove_dup_list([1, 2, 2, 3, 3, 3]))
```

40 — Sort a list without using sort().

```
def bubble_sort(lst):  
    lst = lst.copy()  
    n = len(lst)  
    for i in range(n):  
        for j in range(n - i - 1):  
            if lst[j] > lst[j + 1]:  
                lst[j], lst[j + 1] = lst[j + 1], lst[j]  
    return lst  
  
print(bubble_sort([5, 2, 8, 1]))
```

41 — Find the sum of a list.

```
def list_sum(lst):  
    return sum(lst)  
  
print(list_sum([1, 2, 3, 4]))
```

42 — Find the average of a list.

```
def list_average(lst):  
    return sum(lst) / len(lst)  
  
print(list_average([2, 4, 6]))
```

43 — Merge two lists into one.

```
def merge_lists(l1, l2):
    return l1 + l2

print(merge_lists([1, 2], [3, 4]))
```

44 — Find the common elements between two lists.

```
def common_elements(l1, l2):
    return list(set(l1) & set(l2))

print(common_elements([1, 2, 3], [2, 3, 4]))
```

45 — Find the intersection of two lists.

```
def list_intersection(l1, l2):
    return list(set(l1) & set(l2))

print(list_intersection([1, 2], [2, 3]))
```

46 — Flatten a nested list.

```
def flatten(nested):
    result = []
    for item in nested:
        if isinstance(item, list):
            result.extend(flatten(item))
        else:
            result.append(item)
    return result

print(flatten([1, [2, 3], [4, [5, 6]]]))
```

47 — Count how many times an element appears in a list.

```
def count_occurrences(lst, item):
    return lst.count(item)
```

```
print(count_occurrences([1, 2, 2, 3, 2], 2))
```

48 — Find the missing number in a list from 1 to N.

```
def find_missing(lst, n):  
    return sum(range(1, n + 1)) - sum(lst)  
  
print(find_missing([1, 2, 4, 5], 5))
```

49 — Rotate a list by k positions.

```
def rotate_list(lst, k):  
    k = k % len(lst)  
    return lst[k:] + lst[:k]  
  
print(rotate_list([1, 2, 3, 4, 5], 2))
```

50 — Check if a list is already sorted.

```
def is_sorted(lst):  
    return lst == sorted(lst)  
  
print(is_sorted([1, 2, 3]))
```

Dictionaries & Sets (51–60)

51 — Count word frequency in a sentence using a dictionary.

```
def word_frequency(s):  
    freq = {}  
    for w in s.split():  
        freq[w] = freq.get(w, 0) + 1  
    return freq  
  
print(word_frequency("the cat the dog the cat"))
```

52 — Merge two dictionaries.

```
def merge_dicts(d1, d2):
    return {**d1, **d2}

print(merge_dicts({"a": 1}, {"b": 2}))
```

53 — Find the key with the maximum value.

```
def key_with_max_value(d):
    return max(d, key=d.get)

print(key_with_max_value({"a": 3, "b": 9, "c": 1}))
```

54 — Invert a dictionary (swap keys and values).

```
def invert_dict(d):
    return {v: k for k, v in d.items()}

print(invert_dict({"a": 1, "b": 2}))
```

55 — Remove a key from a dictionary.

```
def remove_key(d, key):
    d = d.copy()
    d.pop(key, None)
    return d

print(remove_key({"a": 1, "b": 2}, "a"))
```

56 — Sort a dictionary by its values.

```
def sort_dict_by_value(d):
    return dict(sorted(d.items(), key=lambda x: x[1]))

print(sort_dict_by_value({"a": 3, "b": 1, "c": 2}))
```

57 — Check if two sets are disjoint.

```
def is_disjoint(s1, s2):
    return s1.isdisjoint(s2)

print(is_disjoint({1, 2}, {3, 4}))
```

58 — Find the union of two sets.

```
def set_union(s1, s2):
    return s1 | s2

print(set_union({1, 2}, {2, 3}))
```

59 — Find the difference between two sets.

```
def set_difference(s1, s2):
    return s1 - s2

print(set_difference({1, 2, 3}, {2, 3}))
```

60 — Convert two parallel lists into a dictionary.

```
def lists_to_dict(keys, values):
    return dict(zip(keys, values))

print(lists_to_dict(["a", "b"], [1, 2]))
```

Functions & Recursion (61–70)

61 — Write a recursive factorial function.

```
def factorial_recursive(n):
    return 1 if n == 0 else n * factorial_recursive(n - 1)

print(factorial_recursive(5))
```

62 — Write a recursive Fibonacci function.

```
def fib_recursive(n):
    return n if n <= 1 else fib_recursive(n - 1) + fib_recursive(n - 2)

print(fib_recursive(7))
```

63 — Sum a list using recursion.

```
def sum_recursive(lst):
    return 0 if not lst else lst[0] + sum_recursive(lst[1:])

print(sum_recursive([1, 2, 3, 4]))
```

64 — Find the power of a number using recursion.

```
def power_recursive(base, exp):
    return 1 if exp == 0 else base * power_recursive(base, exp - 1)

print(power_recursive(2, 5))
```

65 — Implement recursive binary search.

```
def binary_search_recursive(arr, target, low=0, high=None):
    if high is None:
        high = len(arr) - 1
    if low > high:
        return -1
    mid = (low + high) // 2
    if arr[mid] == target:
        return mid
    elif arr[mid] < target:
        return binary_search_recursive(arr, target, mid + 1, high)
    else:
        return binary_search_recursive(arr, target, low, mid - 1)
```

```
print(binary_search_recursive([1, 3, 5, 7, 9, 11], 7))
```

66 — Write a function with default arguments.

```
def greet(name, greeting="Hello"):
    return f"{greeting}, {name}!"

print(greet("Sam"))
```

67 — Write a function using *args and kwargs.

```
def flexible_args(*args, **kwargs):
    return sum(args), kwargs

print(flexible_args(1, 2, 3, x=1))
```

68 — Write a lambda function to square a number.

```
square = lambda x: x * x

print(square(5))
```

69 — Use map() to square every number in a list.

```
squared = list(map(lambda x: x**2, [1, 2, 3, 4]))

print(squared)
```

70 — Use filter() to get only the even numbers from a list.

```
evens = list(filter(lambda x: x % 2 == 0, [1, 2, 3, 4, 5, 6]))

print(evens)
```

File Handling (71–75)

71 — Read a text file line by line.

```
with open("data.txt", "r") as f:
    for line in f:
        print(line.strip())
```

72 — Write data to a text file.

```
with open("output.txt", "w") as f:
    f.write("Hello, file!")
```

73 — Count the number of lines in a file.

```
with open("data.txt", "r") as f:
    count = sum(1 for _ in f)

print(count)
```

74 — Read a CSV file using Python.

```
import csv

with open("data.csv", "r") as f:
    reader = csv.reader(f)
    for row in reader:
        print(row)
```

75 — Append new data to an existing file.

```
with open("output.txt", "a") as f:
    f.write(" More text.")
```

OOP Basics (76–82)

76 — Create a simple class with attributes.

```
class Car:
    def __init__(self, brand, model):
```



```

        self.brand = brand
        self.model = model

c = Car("Toyota", "Corolla")
print(c.brand, c.model)

```

77 — Create a class with a constructor.

```

class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

p = Person("Sam", 25)
print(p.name, p.age)

```

78 — Implement inheritance between two classes.

```

class Animal:
    def speak(self):
        return "..."

class Dog(Animal):
    pass

d = Dog()
print(d.speak())

```

79 — Override a method in a subclass.

```

class Cat(Animal):
    def speak(self):
        return "Meow"

cat = Cat()
print(cat.speak())

```

80 — Create a class with a static method.

```
class MathHelper:
    @staticmethod
    def add(a, b):
        return a + b

print(MathHelper.add(2, 3))
```

81 — Implement a class using a property decorator.

```
class Circle:
    def __init__(self, radius):
        self._radius = radius

    @property
    def area(self):
        return 3.14159 * self._radius ** 2

circ = Circle(2)
print(circ.area)
```

82 — Build a simple Stack class.

```
class Stack:
    def __init__(self):
        self.items = []

    def push(self, item):
        self.items.append(item)

    def pop(self):
        return self.items.pop()

    def is_empty(self):
        return len(self.items) == 0

s = Stack()
s.push(1)
```

```
s.push(2)
print(s.pop())
```



Algorithms (83–92)

83 — Implement bubble sort.

```
def bubble_sort2(arr):
    arr = arr.copy()
    for i in range(len(arr)):
        for j in range(len(arr) - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
    return arr

print(bubble_sort2([5, 1, 4, 2]))
```

84 — Implement selection sort.

```
def selection_sort(arr):
    arr = arr.copy()
    for i in range(len(arr)):
        min_idx = i
        for j in range(i + 1, len(arr)):
            if arr[j] < arr[min_idx]:
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
    return arr

print(selection_sort([5, 1, 4, 2]))
```

85 — Implement insertion sort.

```
def insertion_sort(arr):
    arr = arr.copy()
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
```

```

        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key
    return arr

print(insertion_sort([5, 1, 4, 2]))

```

86 — Implement linear search.

```

def linear_search(arr, target):
    for i, v in enumerate(arr):
        if v == target:
            return i
    return -1

print(linear_search([4, 2, 7, 1], 7))

```

87 — Implement binary search.

```

def binary_search(arr, target):
    low, high = 0, len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1

print(binary_search([1, 3, 5, 7, 9], 5))

```

88 — Check for balanced parentheses in an expression.

```

def is_balanced(s):
    stack = []

```

```

pairs = {')': '(', ']': '[', '}': '{'}
for c in s:
    if c in '([{':
        stack.append(c)
    elif c in ')]}':
        if not stack or stack.pop() != pairs[c]:
            return False
return not stack

print(is_balanced("({[]})"))

```

89 — Implement quicksort.

```

def quicksort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[len(arr) // 2]
    left = [x for x in arr if x < pivot]
    mid = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]
    return quicksort(left) + mid + quicksort(right)

print(quicksort([5, 1, 4, 2, 8]))

```

90 — Implement merge sort.

```

def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    result, i, j = [], 0, 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i]); i += 1
        else:
            result.append(right[j]); j += 1

```

```

        return result + left[i:] + right[j:]

print(merge_sort([5, 1, 4, 2, 8]))

```

91 — Find the nth prime number.

```

def nth_prime(n):
    count, num = 0, 1
    while count < n:
        num += 1
        if all(num % i != 0 for i in range(2, int(num**0.5)
+ 1)):
            count += 1
    return num

print(nth_prime(6))

```

92 — Print all primes up to N using the Sieve of Eratosthenes.

```

def sieve_of_eratosthenes(n):
    is_prime = [True] * (n + 1)
    is_prime[0] = is_prime[1] = False
    for i in range(2, int(n**0.5) + 1):
        if is_prime[i]:
            for j in range(i * i, n + 1, i):
                is_prime[j] = False
    return [i for i, p in enumerate(is_prime) if p]

print(sieve_of_eratosthenes(20))

```



Data Science Warm-Ups (93–100)

93 — Calculate the mean, median, and mode of a list.

```

import statistics as st

```

```
data = [4, 8, 6, 5, 3, 8, 9]
print(st.mean(data), st.median(data), st.mode(data))
```

94 — Calculate standard deviation manually, without a library.

```
def std_dev_manual(lst):
    mean = sum(lst) / len(lst)
    variance = sum((x - mean) ** 2 for x in lst) / len(lst)
    return variance ** 0.5

print(std_dev_manual([2, 4, 4, 4, 5, 5, 7, 9]))
```

95 — Generate random numbers and examine their distribution.

```
import random

nums = [random.randint(1, 100) for _ in range(10)]
print(nums)
```

96 — Load data into a pandas DataFrame and print summary statistics.

```
import pandas as pd

df = pd.DataFrame({"a": [1, 2, 3], "b": [4, 5, 6]})
print(df.describe())
```

97 — Plot a simple line chart with matplotlib.

```
import matplotlib.pyplot as plt

plt.plot([1, 2, 3], [4, 5, 6])
plt.show()
```

98 — Perform a basic groupby and aggregation with pandas.

```
df2 = pd.DataFrame({"category": ["A", "A", "B"], "value":
[10, 20, 30]})
```

```
grouped = df2.groupby("category")["value"].sum()  
print(grouped)
```

99 — Handle missing values in a small pandas DataFrame.

```
df3 = pd.DataFrame({"a": [1, None, 3]})  
filled = df3.fillna(0)  
print(filled)
```

100 — Load a CSV file and filter rows based on a condition using pandas.

```
df4 = pd.read_csv("data.csv")  
filtered = df4[df4["x"] > 2]  
print(filtered)
```